

1. テストを書く目的

- ・信頼性の維持
 - ・健全なコードの維持
 - ・円滑な協業
 - ・リグレッション(後戻り)の回避
- バグをしっかりと捉える
テスト設計から実装案がはまること
テストコードからのコミュニケーション
複雑な構造にならざることを保つ

2. テスト範囲(テストレベル)

- ・静的解析 ... TypeScriptやESLintによる構文解析
- ・単体テスト ... テスト対象モジュールが、期待する出力を返すかどうかをテスト
- ・結合テスト ... 複数モジュールの連動が正常かどうかをテスト
- ・E2Eテスト ... アプリ全体が正常に動くかどうかをテスト

テストの目的(テストタイプ)

- ・機能テスト(インタラクションテスト) ... UIのテスト
- ・非機能テスト(アクセシビリティテスト) ... UXのテスト
- ・リグレッションテスト ... 見た目など、差異を検出するテスト

テスト戦略

- ・アイスクリームコーン型 ... 上層テスト(E2Eなど)が厚い。アンチパターン
- ・テストピラミッド型 ... 下層テストが厚い。手厚く書く
- ・★テストテントトローパー型 ... 結合テストに重点を置き、単体テストと静的解析で補う

テストの実行は、拡張機能で時間短縮できるかも

3. テストファイル置き場に指定はなし (すくなく、ルートディレクトリ内のテスト用フォルダ内が一般的)

- ・テスト結果はアサーションとエラーの組み合わせで表現
- ・条件分岐部分はバグの温床 → 重点的にテストを書き、実行内容の分岐やないテストを付ける
- ・例外処理のハンドリングも、テストにおいて挙動チェックできるよ (toThrow()) 非同期もや
- ・1つのテストケース内でのアサーション回数を指定する記述を加えることで、想定外のテスト成功を防げる

4. モック ... 取得したデータの代替品 (成功バリエーションのみでなく、失敗バリエーションも意図的に用意できる)

- ・スタブ ... 依存先への代替
- ・スパイ ... テスト対象の出力の"記録"

サーバ側の作成

- ・WebAPIではAPI呼び出しする関数の中身を、特定のデータを返す出力装置に替える(モックサーバー)
- ・モック生成関数を導入して、レスポンスを容易に切り替えることが出来る → 例外処理テスト

クライアント側の作成

- ・モック関数を用いて、関数の入出力記録を取ることが出来る
 - その他、時間依存性があるテストでは固定時間を設定したり、共通セリットアップを使用したりする
- 関数の動作は一旦停止させて、特定のロジックの中で呼び出したいものをモックで代替

5. UIコンポーネントテストにおける基本機能は、

- ・データの表示
 - ・ユーザ操作の伝播
 - ・関連APIへの接続
 - ・データの動的な書き換え
- 機能が壊れていないかをチェックする
アクセシビリティがあるかどうか

DOM APIが重要
ex) jsdom, happy-dom

UIコンポーネントテストでは、UIの考え方が重要
テストがUIフレームワークの使用方を表している

フォーカス: <fieldset>
使うかテストはな?

基本的な流れは、DOM API を使ってレンダリング → 要素チェックとアサーション

Tips

- ・ できるだけ単体のコンポーネントテストに持つこと、結合テストは最低限にする
- ・ テストデータのセット等をユーティリティ関数として定義しておく便利

実行されるAPI類
は言語モックで

スナップショットテスト... 指定タイミングでのDOMをスナップショットとして保持し、

→ 水は、テストのとき??
画面は、テストのとき??

テスト実行時にそれと比較すると、想定外のリフレクションを防ぐ

HTML要素に、ロール属性 (role="~") などを設定して、代替性を最適化するのが良い
→ 明示的なロール ⇔ 暗黙のロール

文字通りの意味は無いけど...

6. カバレッジレポート... テスト対象の範囲に関するレポート (網羅率など)

⇒ テストの品質評価の指標のひとつになる

レポート専用のライブラリとあるみたいなので、検討してみると良さそう

7. イベント駆動などのテスト専用コンポーネントを作成するのも1つの方法

・ モックサーバー関連のライブラリがあることも多い (Next.js の Router 等とか)

→ テストの事前準備用コードを抽象化するといい (テスト用データを、スクリプトで構築できるようにするやり方はめっちゃ賢)

・ ネットワークレベルで作成できるモックサーバーライブラリも存在する (リクエストのインターセプト等が可能)

→ 例えば、保存ボタンクリック時の定型的な複数の操作を関数として定義しておくとか (別ファイルで定義とか)

・ コンポーネントの持つ責務には注意 → 親側ビジュアルのテストは書くべきではない、あくまで結合テスト

3. 1. 2. 3. 4. 5. 6. 7. 8. 9. 10.

8. UIコンポーネントエクスプローラー... 非エンジニアとのコンポーネント実装の共有が可能なテスト (Storybook など)

・ Decorator を使って、Context などの階層構造を表現するよう (= すると便利)

・ コンポーネントが Web API に依存している場合、結合テストと同様にモック (MSW) を用意するのがよい

・ Storybook では、Play function を使って、インタラクションテスト (≒ 単体テスト) を行うことができる
アクリビリティテストを導入できる

実行されるAPI類
Storybook 最大のメリット

Test Runner を使って、テストツールとして使用することもできる

Tips

- UIコンポーネントエクスプローラー } Storybook など定義したものを、テストコードでも使いたい
単体テスト、結合テスト } ようなシステムが優れている。(ビジュアル的にも速い的にも)

9. ビジュアルリフレクションテスト... スタイル変化を検知するテスト

ページ画面のキャプチャを撮った画像と比較して、差分を検出する

Storybook (+ StoryCap) と reg-suit の組み合わせが便利、Git コミットすると自動で
変更による差分レポートを生成してくれるような CI を導入できる

10. E2Eテスト... 本物のアプリケーションとほぼ同等の動作のテスト

・ 画面操作による挙動

・ DB サーバ、外部ストレージとの連携 → Docker Compose と連携するといい (勿論 CI も OK)

実行時間が長く、不安定で稀に失敗することもある (Flaky) のに注意

CRUD 操作は、他のテストと干渉することを防ぐため、テールはそのテスト内で完結させるのが良い

日本読者や作